

DAY 6

In PySpark, joins are used to combine data from multiple DataFrames based on a common key or condition, enabling relational data processing in a distributed environment. They are essential for data engineering and analytics tasks where information is spread across different tables.

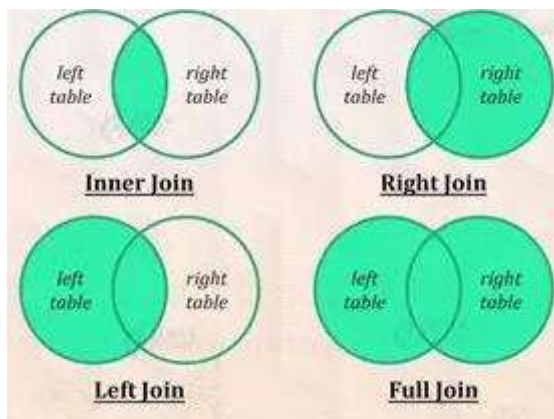
An **inner join** returns only the rows that have matching keys in both DataFrames. If a record does not have a match in both datasets, it is excluded from the result. This type of join is commonly used when only complete and matching data is required for analysis.

A **left join (left outer join)** returns all records from the left DataFrame and the matching records from the right DataFrame. If there is no match, the result will contain null values for the columns from the right DataFrame. This join is useful when the left dataset is the primary reference and we want to retain all its records.

A **right join (right outer join)** is the opposite of a left join. It returns all records from the right DataFrame along with matching records from the left DataFrame. Non-matching rows from the left DataFrame will have null values. This join is less commonly used but helpful when the right dataset is the main reference.

A **full outer join** returns all records from both DataFrames, whether or not they match. When there is no match, missing values are filled with nulls. This type of join is useful when we need a complete view of all data from both sources.

A **cross join** produces a Cartesian product of both DataFrames, meaning every row from the first DataFrame is combined with every row from the second DataFrame. It is generally used only in specific scenarios due to its high computational cost.



DAY 7

Join function and training

DAY 8

Why does rank() skip numbers?

`rank()` skips numbers because it assigns the same rank to rows with equal values (ties), and then advances the next rank by the number of tied rows. For example, if two products share rank 1, the next product will get rank 3 instead of 2. This behavior reflects the actual positional standing in a sorted list where ties occupy multiple positions, so the subsequent rank must account for those occupied spots.

When should you use dense_rank() instead of row_number()?

You should use `dense_rank()` when you want to assign the same rank to tied values while still maintaining a continuous sequence of ranks without gaps. Unlike `row_number()`, which forces a unique sequential number for each row regardless of ties, `dense_rank()` preserves logical grouping of equal values. It's especially useful in scenarios like leaderboards or grouped comparisons where ties should be treated equally but you don't want skipped ranks.

What happens if we don't use orderBy() in a window?

If you don't use `orderBy()` in a window specification for ranking functions, the results become non-deterministic because Spark has no defined way to order the rows. This means the ranking assigned to rows can vary between executions, making the output unreliable. Ranking functions fundamentally depend on a defined order, so omitting `orderBy()` removes the basis for consistent ranking and should be avoided.

DAY 9

Advantages of Apache Superset:

Apache Superset is an open-source modern data exploration and visualization platform designed to be fast, lightweight, and highly scalable. It offers a wide range of benefits for organizations looking to empower users with self-service analytics.

1. Ease of Use Superset provides both a no-code visualization builder and a powerful SQL IDE, making it accessible to non-technical users while still catering to advanced analysts.

Users can drag-and-drop to create charts or write custom SQL queries for more complex needs.

2. **Broad Database Integration** It connects to any SQL-based database, including modern cloud-native and petabyte-scale engines, without requiring an additional ingestion layer. This allows organizations to leverage their existing data infrastructure efficiently.

3. **Rich Visualization Options** Superset comes with 40+ pre-installed visualization types, from simple line charts to advanced geospatial maps. Its plug-in architecture enables developers to add custom visualizations tailored to specific business needs.

4. **Interactive and Customizable Dashboards** Features like cross-filters, drill-to-detail, and drill-by allow deeper data exploration. Dashboards can be customized with CSS templates to match branding, and Jinja templating enables dynamic, interactive filtering.

5. **Performance Optimization** Built-in data caching ensures faster load times for charts and dashboards, improving the user experience even with large datasets.

6. **Semantic Layer and Virtual Datasets** Superset's semantic layer allows SQL data transformations and unified metric definitions. Virtual datasets enable ad-hoc exploration without altering the underlying database schema.

7. **Scalability and Modern Architecture** Its lightweight, modular design supports horizontal scaling and high availability, making it suitable for enterprise-level deployments.

8. **Extensibility and Feature Flags** Superset supports feature flags for enabling experimental or advanced functionalities, allowing teams to adopt new capabilities incrementally.

What is Superset? What is it mainly used for?

Apache Superset is an open-source business intelligence and data visualization tool originally developed by Airbnb. It provides a web-based platform where users can explore data, create visualizations, and build dashboards. Superset is mainly used for analyzing large datasets and presenting insights in a clear, visual format, making it useful for data analysts, engineers, and business teams who want to make data-driven decisions.

Dashboard Features – What types of charts are available?

Superset offers a wide range of chart types to suit different kinds of data analysis. These include basic charts like bar charts, line charts, pie charts, and tables, as well as more advanced options like heatmaps, scatter plots, geographic maps, and time-series visualizations. This variety allows users to represent data in the most meaningful way depending on the use case.

Dashboard Features – Can we add filters and interactivity?

Yes, Superset supports interactive dashboards with built-in filtering options. Users can add filters such as dropdowns, date ranges, and selection tools that allow viewers to interact with the data. For example, clicking on a chart can update other charts in the dashboard, making it easy to drill down into specific insights and explore relationships between data points.

Dashboard Features – How flexible are dashboards (layout, design)?

Dashboards in Superset are quite flexible in terms of layout and design. Users can drag and drop charts, resize them, and arrange them in different grid layouts to create organized and visually appealing dashboards. While it may not offer as much design customization as some premium tools, it still provides enough flexibility for most reporting and analytical needs.

Data Support – What data sources can it connect to?

Superset can connect to a wide range of data sources, including relational databases like MySQL and PostgreSQL, as well as modern cloud data warehouses such as Google BigQuery and Snowflake. It works as a visualization layer, meaning it queries data directly from these sources rather than storing it itself, which makes it scalable and adaptable to different environments.

Ease of Use – Is it easy for beginners or business users?

Superset is relatively easy to use for beginners when performing basic tasks like creating charts or dashboards through its graphical interface. However, for more advanced analysis, users may need some understanding of SQL and data concepts. Business users can still benefit from it, but there may be a small learning curve before they can fully utilize its more powerful features.

Windows functions

In PySpark, window functions like **lag()**, **lead()**, and **sum()** are used for row-wise analytical operations without reducing the number of rows. The **lag()** function returns the previous row's value within a window, while **lead()** returns the next row's value, both useful for comparing sequential data. The **sum()** window function calculates a cumulative or running total over a defined partition and order. These functions are applied using a **Window** specification with **partitionBy()** and **orderBy()** to control grouping and sequence of rows.